

Web Technologies

(Lecture 1: Introduction to Web Technologies + Lecture 2: HTML)

How to use these notes: Each topic is explained in plain, simple words first, then an example is given if the idea is tricky. **Bold** points are the ones most likely to be asked in exams/quizzes. Nothing from the slides has been left out.

PART 1: INTRODUCTION TO WEB TECHNOLOGIES

1. Major Objectives of the Course

This is basically the roadmap of the whole subject — think of it as the "table of contents" for everything you'll learn:

1. **Architecture of Web Applications** — how the pieces of a web app fit together
2. **Client-Side Languages** → HTML, CSS, JavaScript (things that run in the browser)
3. **Server-Side Programming** using Node.js (things that run on the server)
4. **Web Services (REST API)** — how apps talk to each other over the internet
5. **Database Connectivity** with MongoDB
6. **HTTP and DNS Protocol, Domain Settings**
7. **Single Page App Development** using React.js
8. **Web Application Deployment** — putting your app online for the world to use

Memory trick: *"A Client Sends Data Home, Really Deployed"* → **A**rchitecture, **C**lient-side, **S**erver-side, **D**atabase, **H**TTp/**D**NS, **R**eact **S**PA, **D**eployment.

2. Basic Building-Block Terms

Client

- A software/entity that **makes a request** to a server.
- Example: Your web **browser** (Chrome, Firefox) makes a request for a webpage on your behalf. That's why a browser is also called a **user-agent**.

Server

- A software that **serves (answers) requests**.
- **Important point:** Client and server are just *processes* (running programs), not necessarily different physical machines — they **can be on the same computer**. E.g.,

while coding, you often run your own server and your own browser (client) on the same laptop.

Port

- A **port** is a number that **uniquely identifies a process running on a computer**.
- **Easy example:** Imagine a computer is an apartment building. The IP address is the building's street address, and the port number is the apartment/flat number. Many "apartments" (processes/services) can exist in the same "building" (computer), and the port tells the request exactly which apartment to knock on.
- Common examples: Port 80 for HTTP, Port 443 for HTTPS (not explicitly on the slide, but useful to remember).

URL (Uniform Resource Locator)

Example given in the slide:

`http://example.com/search?city=thr&type=electronics`

Breaking it down:

- `http://` → protocol used
- `example.com` → domain/server address
- `/search` → the path/resource being requested
- `?city=thr&type=electronics` → query parameters (extra data sent with the request)

3. More Basic Terms

Protocol

- A **formally defined set of rules** for transmitting, representing, or storing data between two entities.
- Basically an "**agreed-upon behavior**" — both sides must follow the same rulebook to understand each other.
- **Examples:** HTTP, TCP, UDP, DNS.
- **Easy analogy:** Like a language and etiquette used in a phone call — both people must speak the same language and follow calling etiquette (say "hello", wait for a reply) for the conversation to work.

Web Technologies

- The **protocols, languages, libraries, and frameworks** that help build and deploy web apps/services.
- **Examples:** JavaScript, HTTP, DNS, Node.js, Express.

Client-Side Languages

- Languages that run **inside the browser** (on the user's machine).
- **Examples:** HTML, CSS, JavaScript.

Server-Side Languages

- Languages that run **on the server** and generate the output that is sent to the client.
- **Examples:** PHP, C#, Ruby, Java.

Important distinction to remember: Client-side code is visible/runs on the user's device; server-side code runs on a remote machine and the user never sees its raw code — only the final result.

4. Concept of Software Tiers and Layers

This is one of the **most important sections** — architecture questions are commonly asked from here.

What is a "Tier"?

- **Tier** describes the **deployment location** of software components — i.e., *where physically* (which process/computer) a piece of the software lives.

One-Tier Architecture (Monolithic Architecture)

- **All components** — UI, business logic, and database — live **on a single system**.
- Common in **desktop applications** or simple web apps with an embedded database.
- **Examples:**
 - A standalone app like an **MS Access file** (the UI, logic and data are all inside one file).
 - An old-school **PHP script with an embedded SQLite database**.

✓ Advantages

Simple to develop and deploy
Faster performance for small apps

✗

Disadvantages

Not scalable
Hard to maintain

Easy way to remember: One-tier = "everything in one box." Great for something small and personal, terrible once many people need to use it at once.

Two-Tier Architecture (Client–Server Architecture)

The application is split into **two layers**:

1. **Client** — the front end (browser or desktop app) that sends requests.
2. **Server** — the back end that processes requests and talks to the database.

Examples:

- A desktop application with a **separate** database.
- A web application with an **embedded** database.

✓ Advantages

Better scalability than one-tier

Improved security by separating UI from the database

✗ Disadvantages

Limited scalability (heavy load falls directly on the database)

Direct database exposure increases security risk

Three-Tier Architecture (Most Common in Modern Web Apps)

The application is divided into **three parts**:

1. **Presentation Layer (Client)** — the UI: HTML, CSS, JavaScript
2. **Application Layer (Server)** — handles business logic: Node.js, Django, Spring Boot
3. **Data Layer (Database Server)** — stores/retrieves data: MySQL, MongoDB, etc.

Example (this is the classic modern setup): A **React** frontend talks to a **Node.js** backend, which then queries a **MySQL** database.

[React Frontend] --request--> [Node.js Server] --query--> [MySQL Database]

[React Frontend] <---response--- [Node.js Server] <---data--- [MySQL Database]

✓ Advantages

Highly scalable

Better security (database is never directly exposed to the client)

Easier maintenance and upgrades

✗ Disadvantages

More complex development

Requires more resources

Why is this the most popular architecture today? Because each layer can be scaled, maintained, or replaced **independently**. For example, you could switch MySQL for MongoDB without touching your frontend code at all.

N-Tier Architecture

- An **extension** of three-tier architecture where the application is broken into **even more layers/services** (e.g., separate layers for caching, load balancing, multiple microservices, etc.)
- **Preferred because it is easy** to scale, maintain, and update individual pieces without affecting the whole system.

Layer (different from Tier!)

Very important exam point: *Tier* = physical/deployment location. *Layer* = **logical** division of code.

- Layers are **logical divisions** in the code — they *could* all technically run inside **one single application/process**.
- We split code into layers purely for **better maintainability** (organization), not necessarily for separating them onto different machines.

The typical layer flow:

Presentation Layer

↓

Business Logic Layer

↓

Data Access Layer

↓

Database

Easy way to tell Tier vs Layer apart: If you moved a "tier" to a different computer, it would still work the same way (that's the point of tiers). A "layer" is more like organizing your code into neat folders/files inside the same project — it's about **code organization**, not physical location.

5. HTTP Request/Response (Client–Server interaction)

- The slides show diagrams of how a **client sends an HTTP Request** to a server, and the **server sends back an HTTP Response**.
- **Simple mental model:**
 1. Browser (client) types a URL → sends an **HTTP Request** to the server.
 2. Server processes the request (maybe checks a database) → sends back an **HTTP Response** (usually an HTML page, JSON data, etc.).
- This request/response cycle is the **foundation of everything on the web** — every click, form submission, and page load follows this pattern.

6. Service Oriented Architecture (SOA)

Slide note says: "*skip for now, TBD (to be discussed) later*" — but still good to know the basic idea:

- **SOA** = exposing the backend of a system as **different services**, in a way that is **independent of language and platform**.
- Clients communicate with these services instead of directly with the backend code.
- **Benefit:** Makes it easy to **integrate with other systems**, no matter what technology those systems were built with.
- **Example type:** RESTful Services.
- **Microservices** = deploying **each service as a separate application**, making it easier to maintain and scale each one on its own.

Easy analogy: Instead of one giant restaurant kitchen doing everything (cooking, billing, delivery), SOA/microservices is like having separate specialized shops (a bakery, a delivery service, a billing office) that all work together but can be run, fixed, or upgraded independently.

PART 2: HTML (Hyper Text Markup Language)

1. What is HTML?

Breaking the name down word by word (this is exactly how the slide explains it — very likely to be asked as a definition question):

- **Hyper Text** = **Linked Text**. One document is linked to another; the user moves from one page to another using **links**.
- **Markup** = the use of **tags** to "mark up" (label) text, images, etc. So HTML code is essentially made up of tags.
- **Language** = there are **certain rules** for how to write those tags correctly.

File extension: Web pages use .html or .htm. .html is the most common, but **neither extension is strictly required** to make a page work.

2. Web Page Structure (Basic Tags Demo)

Key building blocks of every HTML document:

- **DOCTYPE declaration** — tells the browser which version of HTML the page is written in (usually `<!DOCTYPE html>` for HTML5).
- Core tags: `<html>`, `<head>`, `<body>`, `<h1>`, `<p>`

Anatomy of an HTML Tag

- **Start tag** and **End tag** — e.g., `<p> ... </p>`
- **Tag text/content** — what's written between the start and end tag

- **Tag attribute** — extra information added inside the start tag, e.g., `<p class="intro">`
- **Tag nesting** — putting one tag inside another, e.g., `<p><b>bold text</b></p>`
- **Self-closing tag / Empty tag** — a tag that has **no body/content**, e.g., `<img />`, `<br />`

Golden Rule (very important!): *"Close first what you opened last."* This means tags must close in the **reverse order** they were opened — like nested boxes.

Example of correct nesting:

```
<div>
  <p>
    <b>Hello</b>
  </p>
</div>
```

Here `<b>` was opened last, so it closes first; `<div>` was opened first, so it closes last. If you mix this up (e.g., close `</div>` before `</p>`), the browser can misrender your page.

3. Web Page, Tag, and Attributes

- The **web browser parses (reads)** the HTML code and **renders** (draws) the output on screen.
- HTML code is a set of tags from a **predefined list** (e.g., `p`, `html`, `body`) — each tag has a specific purpose/objective.
- **Attributes** give **extra information** about a tag.
 - Example: `<a href="https://example.com">Click here</a>` — the `href` attribute tells the browser **where the link should go**.
- Tags may be **nested** inside other tags, or may simply contain content (like `<p>Some text</p>`).
- **HTML documents are also called "web pages."**

4. HTML Tags (continued list from the slides)

- `html`, `head`, `title`, `body`
- **Paragraph and Headings** — `<p>` for paragraphs, `<h1>` to `<h6>` for headings (`h1` = biggest/most important, `h6` = smallest)
- **Lines and Line Breaks** — e.g., `<br>` for a line break, `<hr>` for a horizontal line
- **Text Formatting tags:**
 - `<b>` — bold

- o `<i>` — italic
- o `<sub>` — subscript (small text below the line, e.g., in H₂O)
- o `<sup>` — superscript (small text above the line, e.g., in x²)
- o `<pre>` — preformatted text (keeps spacing/line breaks exactly as typed)
- o `<cite>` — marks a citation/reference (e.g., the title of a work)
- o `<abbr>` — marks an abbreviation (can show a tooltip with the full form)
- o `<address>` — marks contact/address information
- o `<q>` — marks a short inline quotation
- **Comments** — text that is **not displayed** on the page, used by developers to leave notes in the code:

```
<!-- comment text here -->
```

Lecturer's note (from slide): Also learn tag **attributes**: `class`, `id`, `style`, `title` — these are used across almost every HTML tag to control styling and identification.

5. Meta Tags

- Meta tags go inside the `<head>` section and give **information about the page itself** rather than content shown directly on the page.
- **Common uses discussed:**
 - o **Page Character Encoding** — tells the browser which character set to use (e.g., UTF-8), so text (including special characters/other languages) displays correctly.
 - o **Meta Tag – Description** — a short summary of the page's content, often shown in **search engine results**.
 - o **Meta Tag – Open Graph** — special meta tags (prefixed `og:`) used by **social media sites** to control how a link looks when shared (title, description, image, URL).
- **Important real-world use:** *Social sites use meta tags to pull the URL, Title, Description, and Image* when you paste a link into WhatsApp, Facebook, Twitter/X, etc. — that preview card is generated from your page's meta tags!

Lecturer's extra note: Other meta info types include `author`, `revised`, `description`, `keywords`, `http-equiv`. The `link` tag also has attributes: `rel`, `type`, `href` (used mainly to link external CSS files, icons, etc.).

6. dir (Direction) Attribute

- Used to set the **text direction** of content.
- Example values: `ltr` (left-to-right — used for English) and `rtl` (right-to-left — used for Arabic, Urdu, etc.)

- **Why it matters:** Without setting `dir="rtl"` correctly, right-to-left languages can display in the wrong reading order.

7. Links (<a> tag)

This is a big topic — links are literally what make "the web" a **web** (interconnected pages)!

- **Purpose:** Links connect one page to another — same domain or different domains.
- 'a' comes from **Anchor** (meaning "link").
- 'href' stands for **Hypertext Reference** — it is **the link target** (where the link goes).
- The **content the user sees and clicks** is called the **Anchor Text** or **Link Text**.
- **target attribute** — often used for **external links** (e.g., `target="_blank"` opens the link in a new tab).
- **title attribute** — used for **screen readers**, or to show a small tooltip describing where the link leads (good for accessibility).

Relative vs Absolute URL

Type	When to use	Example
Relative URL	Linking to another page within the same web application	<code>about.html</code>
Absolute URL	Linking to external sites	<code>https://example.com/about</code>

- To link to **another part of the same page**, we use **fragment identifiers** (the # symbol) together with a matching name/id on the target element.

SEO tip (important, real-world!)

- Anchor text should be **concise, relevant, and informative**.
- Anchor text and title attributes are used in **SEO ranking** — so you should do keyword research to choose the right wording.

Easy example:

```
<a href="https://example.com" target="_blank" title="Visit Example Website">Visit Example</a>
```

Here: a = anchor tag, href = destination, target="_blank" = open in new tab, title = tooltip for accessibility, "Visit Example" = anchor text.

8. img Tag (Images)

Key points from the slide:

- **src attribute** — the path to the image; can be a **relative path** (internal images) or a full URL (**external images**).
- You **can make an image a link** by wrapping it inside an `<a>` tag:

```
<a href="page.html"></a>
```

- **width and height** properties:
 - **Very important UX point:** On a **slow connection**, if you don't specify width/height, the page layout **shifts** once the image finally loads (because the browser didn't reserve space for it) — this is called **Layout Shift**, and it creates a **bad user experience**.
 - You can simulate a slow connection using **Chrome DevTools** to test this.
 - Note: **Images load in parallel** (multiple images can download at the same time).
- **Single image vs multiple images:**
 - You *can* use **one large image** and resize it for icon/thumbnail/normal sizes — but this wastes bandwidth on small displays.
 - Better: use **multiple image sizes** and let the browser pick — this is what the **srcset** attribute is for.
- **Take care of Aspect Ratio** (don't stretch/squash images) and keep **DPI (Dots Per Inch)** **low** for web use (large DPI = unnecessarily heavy file size).
- **Blocking URL / domain issues** — sometimes an image fails to load because its domain/URL is blocked; always check for "image not found" issues.
- **alt attribute** — Provides alternate text for an image.
 - **Good for SEO** (search engines can "read" what the image is about).
 - **Good for screen readers** (helps visually impaired users understand the image).
- **srcset attribute** — lets you provide multiple image versions for different screen sizes/resolutions (this solves the "single image vs multiple images" problem mentioned above).
- **<picture> and <source> tags** — let you specify an image in **multiple formats** (e.g., WebP with a JPG fallback), so the browser picks the best one it supports.

9. Forms

Forms let a website **collect input from the user** — this is one of the most detailed sections, so let's break it down piece by piece.

Basic Form Fields

- **Input text field and password field**
- **<label> tag with for attribute** — connects a label to its input field (clicking the label focuses/selects the input — great for accessibility and usability).

- **Radio buttons** (type="radio") and **Checkboxes** (type="checkbox")
- **Drop-down list** — made using <select> and <option> tags
- **Hidden input field** — an input that is **not shown** to the user but its value is still submitted with the form (useful for passing extra data like an ID).
- **<textarea>** — for multi-line text; has **rows** and **cols** attributes to control its visible size.
- **Button types:**
 - submit — submits the form
 - reset — clears the form back to default values
 - button — a plain, general-purpose button (does nothing on its own unless you add JavaScript)
- You can also **use a button as an input field**.
- **HTML5 new input types:** url, date, email (these give built-in browser validation, e.g., an email field checks for the @ symbol automatically).

Form Attributes

- **action** — the URL where the form data is sent when submitted.
- **method** — how the data is sent. The two main options:

GET

Data is sent **in the URL** (visible, e.g., ?name=ali)
 Limited amount of data
 Good for search/filter (bookmarkable, shareable link)

POST

Data is sent **in the request body** (hidden from the URL)
 Can send larger amounts of data
 Good for sensitive data (passwords, forms with private info)

Easy way to remember: GET = "getting"/fetching info, so it's visible in the address bar (like asking a question out loud). POST = "posting" a private letter — the data travels quietly inside the request, not shown in the URL.

- **Making an already-filled form (update form):**
 - **UX Tip (important, was specifically emphasized):** *Never ask a user to update a property without showing its current value first* — **except for passwords** (for security, you never pre-fill or reveal a password).

Submitting the Form / Values

- Slides show demo screenshots of: submitting a form, submitting with an **empty value**, using the **value** and **required** attributes with HTML5 input types, and viewing the **form output** (what data actually gets sent to the server).
- **Radio and checkbox fields** — demo shows their appearance and what data is submitted when selected.

- **Option List and Text Area** — demo of drop-downs and text boxes, including how to **pre-select an option** and how to give **options a custom value** (the text shown to the user can be different from the actual value sent to the server).

Hidden Input Field (deeper dive)

- A hidden field's value **is still sent to the server on submit**, even though the user never sees it on the page. Commonly used to pass IDs, tokens, or other backend-only data.

Using a Submit Button as an Input Field — important logical rule!

This is a subtle but commonly tested point:

1. **If you do NOT specify a name** for the submit button → **no value for it is sent** to the server at all.
2. **If you DO specify a name** → the word **submit** is sent as the **default value** (unless you set a custom value).
3. **If there are multiple submit buttons** on one form → **only the one that was actually clicked** sends its name and value to the server (the others are ignored).

Easy example:

```
<form action="/save" method="post">
  <button type="submit" name="action" value="save">Save</button>
  <button type="submit" name="action" value="delete">Delete</button>
</form>
```

If the user clicks **Delete**, only `action=delete` is sent to the server — `action=save` is never sent, even though both buttons exist on the same form.

Reset Button and Standard Button

- **Reset button** — clears/resets all form fields back to their original default values.
- **Standard button** (`type="button"`) — does nothing by itself; typically used together with JavaScript for custom actions.

10. Tables and Lists

Tables

- **Table Row** (`<tr>`) and **Table Data** (`<td>`)
- **Attributes:** `border`, `width`
- **rowspan** — makes a cell **span multiple rows** (stretches vertically)

- **colspan** — makes a cell **span multiple columns** (stretches horizontally)
- **Full table structure tags:**
 - o `<caption>` — table title/caption
 - o `<thead>` — table header section
 - o `<tr>` — table row
 - o `<th>` — table header cell (bold, for column titles)
 - o `<tbody>` — the main table body
 - o `<td>` — table data cell (normal cell)
 - o `<tfoot>` — table footer section

Simple example:

```
<table border="1">

  <caption>Student Marks</caption>

  <thead>

    <tr><th>Name</th><th>Marks</th></tr>

  </thead>

  <tbody>

    <tr><td>Ali</td><td>85</td></tr>

  </tbody>

</table>
```

Lists

- **Ordered list** (`<ol>`) — numbered, has a type attribute with options: 1 (numbers), A (uppercase letters), a (lowercase letters), I (uppercase Roman numerals), i (lowercase Roman numerals).
- **Unordered list** (`<ul>`) — bulleted, has a type attribute with options: disc, circle, square, none.
- **List Items** (`<li>`) — used inside both `<ol>` and `<ul>`.

11. Block Elements vs Inline Elements

Very commonly tested concept — make sure this is crystal clear!

Block Elements

- Occupy **100% of the available width** by default.

- Automatically push the next element to a **new row/line**.
- **Examples:** Headings (<h1>—<h6>), Paragraphs (<p>), <div>, , etc.
- **Are sizeable** — you can set width, height, etc. on them.

Inline Elements

- **Examples:** , <a>
- Occupy only as **much space as their content needs** (not the full width).
- **Do not** push other elements to a new line — they fit *inside* the flow of other content (e.g., a sitting inside a <p> or <div>).
- **You cannot apply width/height properties on inline elements** — unless you change their display type (e.g., turn them into block or inline-block using CSS).

Easy analogy:

- Block element = a **brick** — it takes up a whole row on the wall by itself, and the next brick has to start on a new row.
- Inline element = a **word within a sentence** — it just sits comfortably next to other words on the same line without forcing a line break.

12. HTML5 Semantic Tags

- <div> and are **non-semantic** — they don't describe *what kind* of content they hold, just that "here's a generic block/inline container."
- Using **semantic tags instead is good practice, especially for SEO** — because they clearly tell search engines (and other developers) **what the content inside actually represents**.

Common HTML5 Semantic Tags (from the slide):

- <header> — introductory content/branding area of a page or section
- <nav> — navigation links
- <main> — the main, central content of the page
- <section> — a thematic grouping of content
- <article> — a self-contained, independently distributable piece of content (like a blog post)
- <aside> — content tangentially related to the main content (like a sidebar)
- <footer> — footer content (copyright, contact info, etc.)

Why does semantic markup help SEO? Search engines can better understand the *structure and importance* of your content when you use <nav>, <article>, <main>, etc., instead of a page made entirely of unlabeled <div>s.

13. HTML <head> Tag

- Contains **information about the page** itself — not the content users see rendered directly on the page body, but *metadata*, links to external files, icons, styles, and scripts.

Tags found inside <head>:

Tag	Purpose
<meta>	Different types of information about the web page (author, description, keywords, http-equiv, etc.)
<title>	Sets the page title (shown in the browser tab)
<style>	Defines internal/embedded CSS styling
<link>	Links an external CSS file (has attributes rel, type, href)
<script>	Links an external JS file , or contains inline JavaScript code

14. Structured Data (marked "Skip this — covered in SEO Lecture", but good to know)

- In addition to basic on-page SEO, websites can improve SEO further using **Structured Data**.
- **Structured Data** = annotating a webpage using **standard vocabularies** that search engines understand, so machines (like Google's search engine) can **precisely understand** the webpage's content.
- **Common standards:**
 - o JSON-LD (Google's **recommended** format)
 - o MicroData
 - o RDFa
- JSON-LD and MicroData both use a **common vocabulary published at schema.org**.
- **Reference links given in slide:**
 - o Google's guide: developers.google.com/search/docs/guides/intro-structured-data
 - o MicroData tutorial: tutorialspoint.com/html5/html5_microdata.htm
 - o MicroData spec: [w3.org/TR/microdata/](https://www.w3.org/TR/microdata/)

15. Practice Task (given by the lecturer — good for hands-on revision)

The lecturer asked students to explore and practice:

- **<iframe>, <video>, and <audio> tags** — used to embed different types of content (like embedding a YouTube video, or an audio player). Explore their attributes in detail.
- **Table tags:** <table>, <thead>, <tr>, <th>, <td>

- **"Iframes in detail"** — a referenced external resource for deeper study
- **Audio and video tags** — another referenced external resource
- **List tags:** , , , etc.
- **Text tags:** <cite>, <abbr>, <address>, <q> (explore these hands-on in lab)
- **Task:** Make actual **code samples** that illustrate the usage of different attributes of all these tags — hands-on practice, not just theory.

QUICK-REVISION SUMMARY

Introduction to Web Tech:

- Client = requests; Server = serves requests; Port = identifies a process; URL = address of a resource.
- Protocol = agreed rules for communication (e.g., HTTP, DNS).
- **Tier** = physical/deployment location; **Layer** = logical code division (can live in the same tier).
- One-Tier = everything together (simple, not scalable). Two-Tier = client + server. Three-Tier = presentation + application + data layers (most common today). N-Tier = even more layers/services (best scalability/maintainability).
- SOA/Microservices = expose backend as independent, language-agnostic services.

HTML:

- HTML = Hyper Text (linked) + Markup (tags) + Language (rules).
- Golden rule: close tags in reverse order of opening.
- href = link target; alt = accessibility/SEO text for images; srcset/<picture> = responsive images.
- Always set image width/height to avoid layout shift.
- GET (data in URL, visible, small) vs POST (data in body, hidden, larger/sensitive) for forms.
- Submit button rule: no name → nothing sent; name given → submit sent as default value; multiple submit buttons → only the clicked one is sent.
- Never ask a user to update a value without showing the current value (except passwords).
- Block elements (full width, new line) vs Inline elements (content-sized width, same line, no width/height).
- Semantic tags (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>) are better than plain <div>/ for SEO and clarity.
- <head> holds metadata: <meta>, <title>, <style>, <link>, <script>.

FIN !